NOVELVISTA

TRAINING PROGRAM — PRACTICAL WORKBOOK

Git & GitHub Essentials

10 Step-by-Step Case Study Labs and a 50-Question MCQ Assessment

Aligned to the 4-Day, 16-Hour Hands-On Program

10 Case study labs	**50** MCQs	**4 h 45 min** Estimated time	**4** Modules covered

Participant name	
Batch / Date	
Trainer	

October 16, 2026

How to Use This Workbook

This workbook turns the Git & GitHub Essentials syllabus into guided practice. **Part A** has 10 case study labs that build on each other, from the first commit to a team project. Each lab is written step by step, shows the expected output, and has a troubleshooting table, so you can work through it on your own without waiting for help. **Part B** is a 50-question multiple-choice assessment. **Part C** is the answer key and marking scheme.

Time Plan

Times are realistic estimates for a beginner working at class pace. The ten labs alone take about 3 h 55 min.

#	Company	Case study / part	Module (Day)	Time
1	Zomato	Restaurant List: Create a Repository, Commit, Push and Pull	Module 1 and 2 (Day 1 to 2)	20 min
2	Swiggy	Starter Project: Clone a Repository and Copy It into Your Own	Module 3 (Day 3)	25 min
3	Zepto	Dark-Store Inventory: Working with Branches	Module 3 (Day 3)	20 min
4	Nykaa	Offers Page: Your First Pull Request	Module 4 (Day 4)	20 min
5	Swiggy	Order Flow: Branch on Top of Branch on Top of Branch	Module 3 (Day 3)	25 min
6	Paytm	Bill Reminders: Pull Requests with Code Review and Issues	Module 4 (Day 4)	25 min
7	Myntra	Sale Banner: Fixing a Conflict in a Pull Request	Module 3 and 4 (Day 3 to 4)	25 min
8	Razorpay	Payment Records: Reading History and Undoing Mistakes	Module 2 (Day 2)	20 min
9	Ola	Contributor Wall: Fork a Repository and Send a Pull Request	Module 4 (Day 4)	20 min
10	Zepto	Dark-Store Guide: Team Mini Project (Capstone)	Module 4 (Day 4)	35 min
		Part A total: 10 case study labs		**3 h 55 min**
		Part B: 50 MCQs (about 1 minute each)		**50 min**
		Workbook total		**4 h 45 min**

How to Work Through a Lab

1. Read the **Business scenario** and the **Learning Objectives** first (one minute).
2. Do the tasks in order. Type the commands yourself instead of copying them, and replace every placeholder in angle brackets such as `<username>`.
3. After each task compare your screen with the **Output** box.

© Akshat Gupta — All Rights Reserved

4. At the end compare your result with **Expected result**. If it does not match, use the **If something goes wrong** table.
5. Keep the evidence listed under **Deliverables** and answer the reflection questions in your own words.

Conventions Used

Element	Meaning
Dark code block	Commands to type exactly as shown. Lines starting with # are comments.
Output block	What you should expect to see. Hashes, dates and paths will differ on your machine.
`<username>`	Replace the angle-bracket placeholder with your own value.
Blue box	A concept or tip that explains why a step matters.
Orange box	A warning: the step is risky or needs special care.
Green box	Expected result: compare it with your own to confirm the lab is complete.

Commit Message Convention

Use a short type prefix and a specific summary: `feat:` new capability, `fix:` bug fix, `docs:` documentation, `chore:` housekeeping. Example: `feat: add third restaurant`. Vague messages such as `changes` or `asdf` are not accepted in any lab.

© Akshat Gupta — All Rights Reserved

Connecting Your Computer to GitHub (Do This Once)

Do the four steps on this page before Case Study 1. They take about 15 minutes. After this you can push and pull without further setup.

Step 1: Install Git and create your lab folder

1. Install Git from git-scm.com (Windows users: install **Git for Windows** and then always use **Git Bash** as your terminal, not Command Prompt). macOS and Linux users use the normal Terminal.
2. Open the terminal and run the commands below. `git --version` should print a version number.

```
git --version
mkdir ~/git-labs
cd ~/git-labs
pwd
```

`~/git-labs` is the one folder where you will create every lab project. Each lab starts with `cd ~/git-labs`.

Step 2: Tell Git who you are

Git stores your name and e-mail in every commit. Use the same e-mail as your GitHub account.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global --list
```

Step 3: Create your GitHub account

1. Go to github.com and click **Sign up**. Choose a username, confirm your e-mail address from the mail GitHub sends you.
2. Turn on two-factor authentication: profile picture (top right), **Settings**, **Password and authentication**, **Enable two-factor authentication**.

Step 4: Choose how your laptop signs in to GitHub

GitHub no longer accepts your account password when Git pushes your code. Choose ONE of the two options below. **Option A is the simplest and recommended for this course.**

Option A (recommended): HTTPS with a Personal Access Token

1. On GitHub click your profile picture, **Settings**, scroll down the left menu and click **Developer settings**.
2. Click **Personal access tokens**, **Tokens (classic)**, then **Generate new token**, **Generate new token (classic)**.
3. Note: `git-training`. Expiration: 30 days. Tick the box **repo**. Click **Generate token**.
4. Copy the token now (it starts with `ghp_`) and keep it in a safe place. GitHub will not show it again.

© Akshat Gupta — All Rights Reserved

5. The first time you run `git push` (in Case Study 1) Git asks you to sign in. Either choose **Sign in with your browser** if a window opens, or enter your GitHub username and paste the **token as the password**. Git remembers it afterwards.

Option B: SSH key (no password or token needed later)

1. Create a key. Press **Enter** three times to accept the defaults:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

1. Show the public key and copy the whole line (it starts with `ssh-ed25519`):

```
cat ~/.ssh/id_ed25519.pub
```

1. On GitHub open **Settings**, **SSH and GPG keys**, **New SSH key**. Title: `Training laptop`. Key type: **Authentication Key**. Paste the line and click **Add SSH key**.
2. Test the connection. The first time, type `yes` when asked to continue:

```
ssh -T git@github.com
```

```
OUTPUT
Hi <username>! You've successfully authenticated, but GitHub does not provide shell
access.
```

In every lab the repository URLs are written in the HTTPS form. **If you chose SSH, use the SSH form instead:**

Form	Example
HTTPS	`https://github.com/<username>/zomato-restaurants.git`
SSH	`git@github.com:<username>/zomato-restaurants.git`

Check before you continue

`git config --global --list` shows your name and e-mail.

You have your token saved (Option A) or `ssh -T git@github.com` says you authenticated (Option B).

The folder `~/git-labs` exists.

© Akshat Gupta — All Rights Reserved

Part A — Case Study Labs

The ten labs follow a simple path. First you learn the everyday cycle (Case Study 1), then cloning and copying (2), branches (3), your first Pull Request (4), a chain of branches built from a diagram (5), reviewing Pull Requests with Issues (6), fixing conflicts in a Pull Request (7), undoing mistakes (8), contributing through a fork (9), and finally a team capstone (10).

Lab	Company	What you will practise	Syllabus module
1	Zomato	Create a repo, commit, push, pull	Module 1 and 2
2	Swiggy	Clone a repo, copy the code into your own repo	Module 3
3	Zepto	Create, switch, push and merge branches	Module 3
4	Nykaa	Your first Pull Request	Module 4
5	Swiggy	Branches on top of branches (diagram)	Module 3
6	Paytm	Issues, Pull Request review in pairs	Module 4
7	Myntra	Fix a conflict in a Pull Request	Module 3 and 4
8	Razorpay	Read history, undo mistakes safely	Module 2
9	Ola	Fork, clone and contribute	Module 4
10	Zepto	Capstone: complete team workflow	Module 4

Working rules

Create every project inside `~/git-labs`, one folder per lab. Never run `git init` inside another repository.

Run `git status` often. It tells you where you are and what to do next.

Read an error message to the end: Git usually tells you how to fix it. Use the table at the end of each lab and Appendix B.

© Akshat Gupta — All Rights Reserved

Case Study 1 — Restaurant List: Create a Repository, Commit, Push and Pull

Company	Module	Time	Level
Zomato	Module 1 and 2 (Day 1 to 2)	20 minutes	Beginner

> **Business scenario — Zomato**
> You have joined Zomato as an intern. Your first job is to keep a small restaurant list in a Git repository, store it on GitHub, and learn the everyday cycle every developer follows: change a file, commit, push to GitHub, and pull changes made by others.

Learning Objectives

- Create an empty repository on GitHub and a project on your laptop
- Run the basic cycle: edit, git add, git commit
- Connect the laptop project to GitHub and push
- Make a change on GitHub and bring it back with git pull

Practical Tasks

Task 1.1 — Create an empty repository on GitHub (3 min)

1. Sign in to github.com. Click the **+** icon (top right) and choose **New repository**.
2. Repository name: `zomato-restaurants`. Choose **Public**.
3. Leave **Add a README file** UNTICKED. Do not choose any other options.
4. Click **Create repository**. Keep this page open: you will need the repository URL.

Task 1.2 — Create the project on your laptop and make the first commit (5 min)

Open your terminal (Git Bash on Windows) and run these commands one block at a time.

```
cd ~/git-labs
mkdir zomato-restaurants
cd zomato-restaurants
git init
```

Now create two files:

```
echo "Restaurant list" > restaurants.txt
echo "1. Spice Garden - Koramangala" >> restaurants.txt
echo "2. Dosa Corner - Indiranagar" >> restaurants.txt
echo "# Zomato Restaurants" > README.md
```

Check the state, stage the files, check again, then commit:

© Akshat Gupta — All Rights Reserved

```
git status
git add .
git status
git commit -m "feat: add first two restaurants"
git log --oneline
```

```
OUTPUT
[main (root-commit) 1a2b3c4] feat: add first two restaurants
 2 files changed, 4 insertions(+)
1a2b3c4 feat: add first two restaurants
```

Notice how `git status` changed from red Untracked files to green Changes to be committed after `git add .`.

Task 1.3 — Connect to GitHub and push (4 min)

Replace `<username>` with your GitHub username.

```
git remote add origin https://github.com/<username>/zomato-restaurants.git
git remote -v
git branch -M main
git push -u origin main
```

> **Sign-in prompt**
> The first time you push, a window or prompt asks you to sign in to GitHub. Follow the method you set up on the page 'Connecting Your Computer to GitHub' (browser sign-in or token as the password, or SSH if you chose it).

Refresh your repository page on GitHub. You should now see both files and the commit message.

Task 1.4 — Change a file on your laptop and push again (3 min)

```
echo "3. Tandoor Nights - Jayanagar" >> restaurants.txt
git status
git diff
git add restaurants.txt
git commit -m "feat: add third restaurant"
git push
```

`git diff` showed the new line in green before you staged it. This time `git push` needs no extra words, because `-u` already linked your branch to GitHub.

Task 1.5 — Change a file on GitHub and pull it to your laptop (5 min)

1. On GitHub open your repository and click **restaurants.txt**.
2. Click the **pencil icon** (Edit this file). Add a new last line: `4. Chai Corner - HSR Layout`.
3. Click **Commit changes...**, type the message `feat: add fourth restaurant (from GitHub)` and click **Commit changes**.

Your laptop does not know about this change yet. Check, then pull:

© Akshat Gupta — All Rights Reserved

```
cat restaurants.txt
git pull
cat restaurants.txt
git log --oneline
```

```
OUTPUT
Updating 5d6e7f8..9a0b1c2
Fast-forward
 restaurants.txt | 1 +
 1 file changed, 1 insertion(+)
```

Expected result — check your work

GitHub shows 3 commits and both files.

`restaurants.txt` on your laptop lists all four restaurants after `git pull`.

`git log --oneline` on your laptop shows the same 3 commits as GitHub.

If Something Goes Wrong

What you see	What to do
Author identity unknown / Please tell me who you are	Run `git config --global user.name "Your Name"` and `git config --global user.email "you@example.com"`, then repeat the commit.
Authentication failed / password authentication was removed	Do not use your GitHub password. Use browser sign-in or your token (see 'Connecting Your Computer to GitHub').
error: src refspec main does not match any	You have no commit yet. Run `git add .` and `git commit -m "..."`, then push again.
remote origin already exists	Fix the address with `git remote set-url origin <correct-url>`.
fatal: not a git repository	You are in the wrong folder. Use `cd ~/git-labs/zomato-restaurants`.

Deliverables

☐	Your repository URL
☐	Screenshot of the GitHub page showing the files and 3 commits
☐	Output of `git log --oneline` from your laptop

Reflection Questions

- What is the difference between `git commit` and `git push`?
- What did `git pull` do, and why did your laptop need it?
- Which of these live only on GitHub, and which live on your laptop: the first commit, the fourth restaurant?

© Akshat Gupta — All Rights Reserved

Case Study 2 — Starter Project: Clone a Repository and Copy It into Your Own

Company	Module	Time	Level
Swiggy	Module 3 (Day 3)	25 minutes	Beginner

> **Business scenario — Swiggy**
> Your Swiggy team lead shares a public starter project and says: "Make your own copy in your GitHub account so you can experiment without touching the original." You will clone the starter project, create your own repository, copy the code across, and push it. You will practise git clone twice and see why the new repository has a different history.

Learning Objectives

- Clone a public GitHub repository
- Create your own empty repository and clone it
- Copy project files from one local repository to another
- Commit and push the copied code to your own repository

Practical Tasks

Task 2.1 — Clone the public starter repository (4 min)

You will use GitHub's public practice repository `octocat/Spoon-Knife`. (If your trainer gives you a different repository, use that URL instead.)

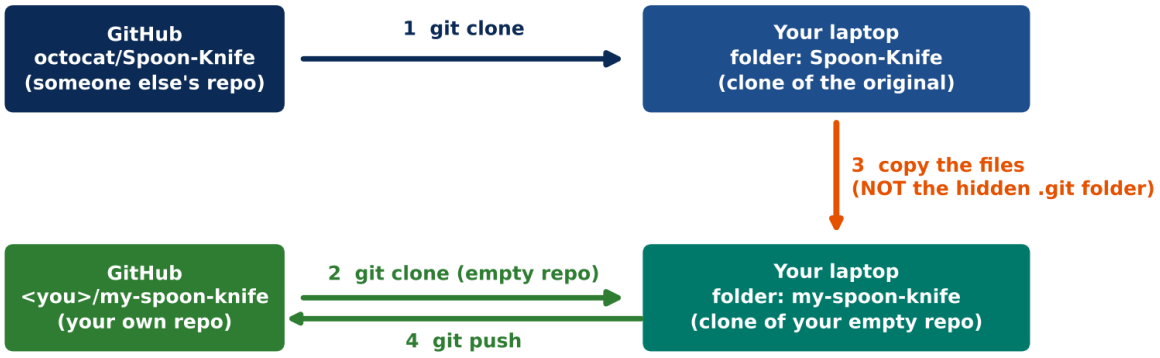


Figure 1: The four moves of this lab

```
cd ~/git-labs
git clone https://github.com/octocat/Spoon-Knife.git
cd Spoon-Knife
ls
git log --oneline
git remote -v
```

© Akshat Gupta — All Rights Reserved

`ls` shows the project files (README.md, index.html, styles.css). `git remote -v` shows that `origin` points to **octocat**'s repository. You can read it, but you cannot push to it because it belongs to someone else.

Task 2.2 — Create your own empty repository on GitHub (3 min)

1. On GitHub click **+**, **New repository**.
2. Name: `my-spoon-knife`. Choose **Public**. Leave **Add a README file** UNTICKED. Click **Create repository**.

Task 2.3 — Clone your own empty repository (3 min)

```
cd ~/git-labs
git clone https://github.com/<username>/my-spoon-knife.git
cd my-spoon-knife
git status
```

```
OUTPUT
warning: You appear to have cloned an empty repository.
On branch main

No commits yet
```

The warning is normal because the repository has no files yet.

Task 2.4 — Copy the code from the first project into your project (5 min)

```
cd ~/git-labs
cp -r Spoon-Knife/* my-spoon-knife/
ls my-spoon-knife
```

Prefer File Explorer or Finder? Open the `Spoon-Knife` folder, select the **visible files** (README.md, index.html, styles.css), copy them and paste them into the `my-spoon-knife` folder.

> **Do NOT copy the hidden .git folder**
> The `.git` folder holds the history and the link to octocat's repository. If you copy it, your project would still point to the original repository and your push would fail. The `*` in the command above skips hidden items, which is exactly what we want.

Task 2.5 — Commit and push to your own repository (4 min)

```
cd my-spoon-knife
git status
git add .
git commit -m "chore: copy starter files from Spoon-Knife"
git push -u origin main
```

© Akshat Gupta — All Rights Reserved

Sign-in prompt
The first time you push, a window or prompt asks you to sign in to GitHub. Follow the method you set up on the page 'Connecting Your Computer to GitHub' (browser sign-in or token as the password, or SSH if you chose it).

Refresh `my-spoon-knife` on GitHub. The files should now be there.

Task 2.6 — Make the project yours (4 min)

```
echo "" >> README.md
echo "Customised by <your name>" >> README.md
git add README.md
git commit -m "docs: add my name to README"
git push
```

Open the README on GitHub and check that your name appears at the bottom.

Task 2.7 — Compare the two histories (2 min)

```
cd ~/git-labs/Spoon-Knife
git log --oneline
cd ~/git-labs/my-spoon-knife
git log --oneline
```

The original project has many commits; yours has only 2. Copying files does not copy history. Write down why.

Another way (keeps the full history)
Inside the cloned `Spoon-Knife` folder you could instead run `git remote set-url origin https://github.com/<username>/my-spoon-knife.git` and then `git push -u origin main`. That uploads the original history to your own repository. In this lab we copied the files so you could see both repositories side by side.

Expected result — check your work
Your `my-spoon-knife` repository on GitHub contains README.md, index.html and styles.css.
`git remote -v` inside `my-spoon-knife` shows YOUR repository, not octocat's.
Your README ends with the line 'Customised by ...'.

If Something Goes Wrong

What you see	What to do
destination path 'Spoon-Knife' already exists	You already cloned it. Use `cd Spoon-Knife` and continue, or delete the folder and clone again.
cp: cannot stat 'Spoon-Knife/*'	You are in the wrong folder. Run `cd ~/git-labs` first and check with `ls`.

© Akshat Gupta — All Rights Reserved

What you see	What to do
remote: Permission denied / 403 when pushing	You pushed from the wrong folder or copied the .git folder. Delete `my-spoon-knife`, clone it again, copy the files again (without .git).
error: failed to push some refs	Run `git branch` to check you are on `main`. If it says `master`, run `git branch -M main` and push again.

Deliverables

☐	URL of your `my-spoon-knife` repository
☐	Screenshot of its file list on GitHub
☐	Output of `git remote -v` from inside `my-spoon-knife`
☐	Written answer: why do the two histories differ?

Reflection Questions

- What is the difference between cloning a repository and copying its files?
- Why were you able to push to `my-spoon-knife` but not to `Spoon-Knife`?
- Which folder on your laptop contains a hidden `.git` folder?

© Akshat Gupta — All Rights Reserved

Case Study 3 — Dark-Store Inventory: Working with Branches

Company	Module	Time	Level
Zepto	Module 3 (Day 3)	20 minutes	Beginner

> **Business scenario — Zepto**
> Zepto's dark-store app must always work on the main branch. Whenever a developer builds something new, they do it on a separate branch so unfinished work cannot break main. You will build a low-stock alert on its own branch, watch the files appear and disappear as you switch branches, and then merge the finished work into main.

Learning Objectives

- Create a branch and switch between branches
- Commit on a branch without changing main
- Push a branch to GitHub
- Merge a branch into main and delete it

Practical Tasks

Task 3.1 — Set up the project (same steps as Case Study 1) (4 min)

1. On GitHub create an empty **public** repository named `zepto-store` (no README).
2. On your laptop run the commands below, replacing `<username>`.

```
cd ~/git-labs
mkdir zepto-store
cd zepto-store
git init
echo "Zepto Store Inventory" > store.txt
echo "milk: 20" >> store.txt
echo "bread: 15" >> store.txt
echo "eggs: 30" >> store.txt
git add .
git commit -m "feat: add store inventory"
git remote add origin https://github.com/<username>/zepto-store.git
git branch -M main
git push -u origin main
```

Task 3.2 — Create a branch and look around (3 min)

```
git branch
git switch -c feature/low-stock-alert
git branch
git status
```

`git branch` lists your branches; the one with a `*` is the branch you are on. `git switch -c` creates a new branch and moves you onto it.

© Akshat Gupta — All Rights Reserved

Task 3.3 — Work on the branch (4 min)

```
echo "Alert when any item is below 10 units" > alerts.txt
git add alerts.txt
git commit -m "feat: add low stock alert rule"

echo "Low stock threshold: 10" >> store.txt
git commit -am "feat: add threshold to store file"

git log --oneline
```

Now prove that the branch keeps your work separate from main:

```
git switch main
ls
git switch feature/low-stock-alert
ls
```

On `main` the file `alerts.txt` is gone; on the feature branch it is back. Git swaps the files for you.

Task 3.4 — Push the branch to GitHub (3 min)

```
git push -u origin feature/low-stock-alert
```

On GitHub click the branch drop-down (it says **main**). You should see both branches. Select the feature branch and open `alerts.txt`.

Task 3.5 — Merge the branch into main (4 min)

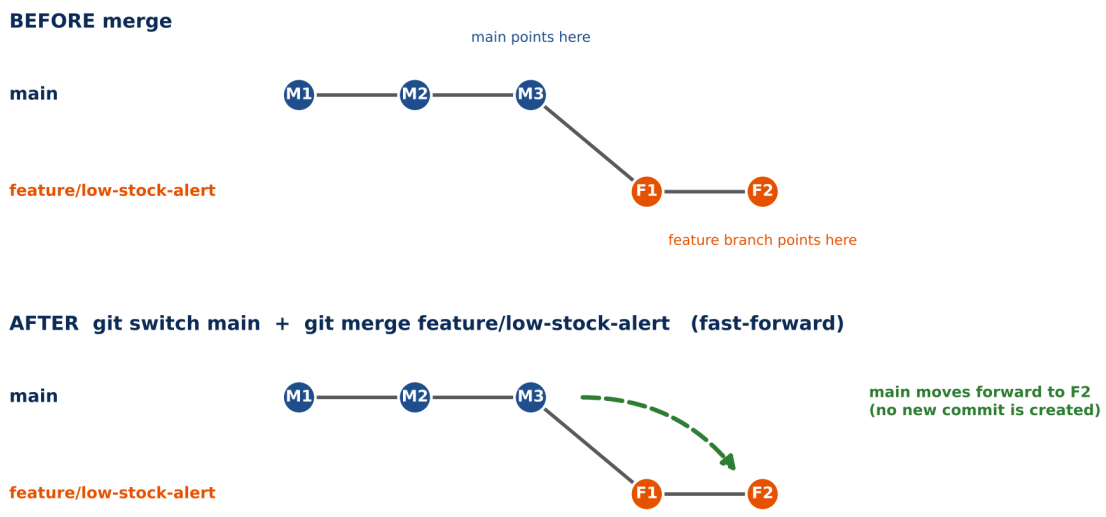


Figure 2: What a fast-forward merge does

```
git switch main
git merge feature/low-stock-alert
git log --oneline --graph --all
git push
```

© Akshat Gupta — All Rights Reserved

```
OUTPUT
Updating 3f2a9c1..8d7e6f5
Fast-forward
 alerts.txt | 1 +
 store.txt  | 1 +
 2 files changed, 2 insertions(+)
```

You merge INTO the branch you are standing on. That is why we switched to `main` first.

Task 3.6 — Delete the finished branch (2 min)

```
git branch -d feature/low-stock-alert
git push origin --delete feature/low-stock-alert
git branch -a
```

Expected result — check your work

`main` contains `store.txt` with the threshold line and `alerts.txt`.

`git log --oneline --graph --all` shows a straight line of commits.

GitHub shows only the `main` branch after the cleanup.

If Something Goes Wrong

What you see	What to do
error: Your local changes would be overwritten by checkout	Commit first: `git add .` then `git commit -m "..."`, and then switch branch.
fatal: a branch named '...' already exists	Do not use `-c`. Just run `git switch <branch-name>`.
Already up to date (but you expected changes)	You are probably on the wrong branch. Run `git branch` to check, switch to `main`, and merge again.
error: The branch '...' is not fully merged	You have not merged it yet. Merge it into main first, then delete with `-d`.

Deliverables

☐	Repository URL
☐	Output of `git log --oneline --graph --all` after the merge
☐	Screenshot of GitHub's branch list while the feature branch existed

Reflection Questions

- Why did `alerts.txt` disappear when you switched to `main`?
- Why did we run `git switch main` before `git merge`?
- Why is it safer to build new features on a branch?

© Akshat Gupta — All Rights Reserved

Case Study 4 — Offers Page: Your First Pull Request

Company	Module	Time	Level
Nykaa	Module 4 (Day 4)	20 minutes	Beginner

> **Business scenario — Nykaa**
> At Nykaa nobody pushes straight to main. Every change goes through a Pull Request (PR) so that it can be looked at before it is merged. You will make a change on a feature branch, push it, open a Pull Request on GitHub, merge it from the browser, and bring the result back to your laptop.

Learning Objectives

- Explain the Pull Request workflow
- Open a Pull Request from a feature branch
- Read the Conversation, Commits and Files changed tabs
- Merge a Pull Request and update your local main

Practical Tasks

Task 4.1 — Create the repository and the first file (4 min)

1. On GitHub create a **public** repository `nykaa-offers`. This time tick **Add a README file**. Click **Create repository**.
2. Click the green **Code** button, make sure **HTTPS** is selected and copy the URL.

```
cd ~/git-labs
git clone https://github.com/<username>/nykaa-offers.git
cd nykaa-offers
echo "NYKAA OFFERS" > offers.txt
echo "Offer 1: 20% off on skincare" >> offers.txt
git add offers.txt
git commit -m "feat: add offers file"
git push
```

Task 4.2 — Create a feature branch and make a change (4 min)

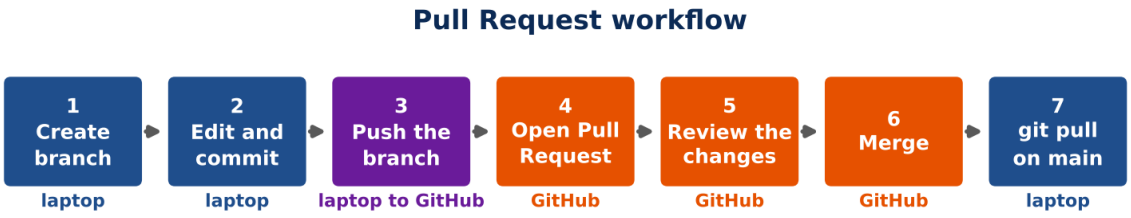


Figure 3: The Pull Request workflow

© Akshat Gupta — All Rights Reserved

```
git switch -c feature/diwali-offer
echo "Offer 2: Diwali sale - flat 30% off on makeup" >> offers.txt
git add offers.txt
git commit -m "feat: add Diwali makeup offer"
```

Task 4.3 — Push the branch (2 min)

```
git push -u origin feature/diwali-offer
```

Open your repository page on GitHub and refresh. A yellow banner appears: **feature/diwali-offer had recent pushes** with a button **Compare & pull request**.

Task 4.4 — Open the Pull Request (4 min)

1. Click **Compare & pull request**. (No banner? Open the **Pull requests** tab, click **New pull request**, set base: `main` and compare: `feature/diwali-offer`.)
2. Check the two drop-downs: **base: main** on the left and **compare: feature/diwali-offer** on the right.
3. Title: `Add Diwali makeup offer`. Description: `Adds Offer 2 to offers.txt.`
4. Click **Create pull request**.

Task 4.5 — Look at the PR and merge it (3 min)

1. Click the **Commits** tab to see your commit, then the **Files changed** tab to see the added line in green.
2. Return to the **Conversation** tab and click **Merge pull request**, then **Confirm merge**.
3. Click **Delete branch** to tidy up.

Task 4.6 — Update your laptop (3 min)

```
git switch main
git pull
cat offers.txt
git log --oneline --graph
git branch -d feature/diwali-offer
```

Your local `main` now contains Offer 2, because the merge happened on GitHub and `git pull` brought it down.

> **Expected result — check your work**
> The Pull Request shows a purple **Merged** badge.
> `offers.txt` on `main` (GitHub and laptop) lists both offers.
> The branch `feature/diwali-offer` is deleted.

© Akshat Gupta — All Rights Reserved

If Something Goes Wrong

What you see	What to do
No yellow banner	Open the Pull requests tab, click New pull request and choose your branch in the compare drop-down.
The merge button says 'This branch has conflicts'	Two people changed the same line. Do Case Study 7 to learn how to fix it.
git pull says 'Already up to date' but the file has no Offer 2	You are on the feature branch. Run `git switch main` and pull again.
PR compares the wrong branches	Change the drop-downs at the top of the PR page: base must be `main`.

Deliverables

☐	Link to your merged Pull Request
☐	Screenshot of the Files changed tab
☐	Output of `git log --oneline --graph`

Reflection Questions

- What is the advantage of merging through a Pull Request instead of pushing to main?
- In which tab can a reviewer see exactly which lines changed?
- Why did you need `git pull` after merging on GitHub?

© Akshat Gupta — All Rights Reserved

Case Study 5 — Order Flow: Branch on Top of Branch on Top of Branch

Company	Module	Time	Level
Swiggy	Module 3 (Day 3)	25 minutes	Beginner to Intermediate

> **Business scenario — Swiggy**
> Swiggy's order flow has four pages that depend on each other: Menu, Cart, Payment and Order Confirmation. The Cart page needs the Menu page, Payment needs Cart, and Confirmation needs Payment. So the team builds each page on a branch that starts from the previous branch. You will build this chain by following the diagram, then merge it back into main in order.

Learning Objectives

- Create a branch from another feature branch (not from main)
- Understand that a new branch contains everything from the branch it started from
- Push several branches to GitHub in one command
- Merge a chain of branches into main in the right order

Practical Tasks

Task 5.1 — Set up the project (4 min)

1. On GitHub create an empty **public** repository `swiggy-order-flow` (no README).

```
cd ~/git-labs
mkdir swiggy-order-flow
cd swiggy-order-flow
git init
echo "# Swiggy Order Flow" > README.md
echo "Base project" > orderflow.txt
git add .
git commit -m "chore: start order flow project"
git remote add origin https://github.com/<username>/swiggy-order-flow.git
git branch -M main
git push -u origin main
```

© Akshat Gupta — All Rights Reserved

Task 5.2 — Study the diagram (2 min)

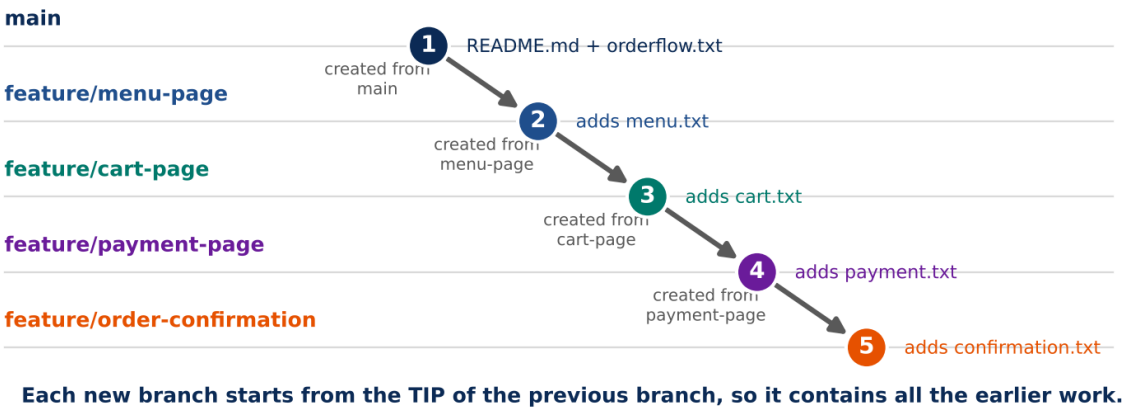


Figure 4: Four branches, each created from the previous one

Read the diagram from top to bottom, then answer in your notes: (a) How many branches will exist at the end, including main? (b) Which branch is `feature/payment-page` created from? (c) Will `feature/cart-page` contain `menu.txt`?

Task 5.3 — Build the chain of branches (10 min)

Branch 1 starts from `main`. Run:

```
git switch -c feature/menu-page
echo "Menu page: show restaurant menu" > menu.txt
git add menu.txt
git commit -m "feat: add menu page"
```

Branch 2 starts from branch 1. Do NOT switch back to main; create it right here:

```
git switch -c feature/cart-page
echo "Cart page: show selected items and total" > cart.txt
git add cart.txt
git commit -m "feat: add cart page"
```

Branch 3 starts from branch 2:

```
git switch -c feature/payment-page
echo "Payment page: choose UPI or card" > payment.txt
git add payment.txt
git commit -m "feat: add payment page"
```

Branch 4 starts from branch 3:

```
git switch -c feature/order-confirmation
echo "Confirmation page: show order number and ETA" > confirmation.txt
git add confirmation.txt
git commit -m "feat: add order confirmation page"


git branch
git log --oneline --graph --all
```

© Akshat Gupta — All Rights Reserved

```
OUTPUT
* 9c8b7a6 (HEAD -> feature/order-confirmation) feat: add order confirmation page
* 5e4d3c2 (feature/payment-page) feat: add payment page
* 1b2a3c4 (feature/cart-page) feat: add cart page
* 7f6e5d4 (feature/menu-page) feat: add menu page
* 0a1b2c3 (origin/main, main) chore: start order flow project
```

Task 5.4 — Prove that each branch contains the earlier work (3 min)

```
git switch main
ls
git switch feature/menu-page
ls
git switch feature/cart-page
ls
git switch feature/payment-page
ls
git switch feature/order-confirmation
ls
```

Fill in the table with the files you see (apart from README.md and orderflow.txt):

Branch	Extra files you see with ls
main	
feature/menu-page	
feature/cart-page	
feature/payment-page	
feature/order-confirmation	

Task 5.5 — Push all branches to GitHub (2 min)

```
git push -u origin --all
```

On GitHub open the branch drop-down. You should see 5 branches. Click **feature/cart-page** and notice that it contains `menu.txt` and `cart.txt`.

Task 5.6 — Merge the chain into main, in order (4 min)

```
git switch main
git merge feature/menu-page
git merge feature/cart-page
git merge feature/payment-page
git merge feature/order-confirmation
git log --oneline --graph
git push
```

Each merge says **Fast-forward** because every branch simply continues from the previous one. `ls` on `main` now shows all four page files.

© Akshat Gupta — All Rights Reserved

Shortcut to remember

Because `feature/order-confirmation` already contains all the earlier branches, merging only that one branch into main would have brought in all four pages.

Expected result — check your work

`ls` on main shows menu.txt, cart.txt, payment.txt and confirmation.txt.

`git log --oneline --graph` shows one straight line of 5 commits.

GitHub shows 5 branches before cleanup and the updated main afterwards.

If Something Goes Wrong

What you see	What to do
The new branch is missing the earlier files	You created it from main by mistake. Switch to the previous feature branch, then run `git switch -c <new-branch>` again (delete the wrong one with `git branch -D <wrong-branch>`).
I mistyped a branch name	Rename it: `git branch -m <old-name> <new-name>`.
Not sure which branch I am on	Run `git branch`. The branch with a `*` is the current branch.
error: failed to push some refs	Run `git push -u origin --all` again from inside the project folder, after your commits are done.

Deliverables

☐	Repository URL
☐	Completed table of files per branch
☐	Output of `git log --oneline --graph --all` before merging
☐	Answers to the three diagram questions

Reflection Questions

- Why does `feature/payment-page` contain `menu.txt` even though you never added it there?
- What would happen if you had created `feature/cart-page` from main instead?
- In which order must the branches be merged, and why?

© Akshat Gupta — All Rights Reserved

Case Study 6 — Bill Reminders: Pull Requests with Code Review and Issues

Company	Module	Time	Level
Paytm	Module 4 (Day 4)	25 minutes	Beginner to Intermediate

> **Business scenario — Paytm**
> At Paytm every change starts as an Issue, is built on a branch, and is merged only after a teammate reviews the Pull Request. You will work in pairs. Learner A is the Author (writes the change) and Learner B is the Reviewer (checks it). If time remains, swap roles and repeat. If you work alone, ask your trainer for a partner or use a second GitHub account for the Reviewer.

Learning Objectives

- Create a GitHub Issue and link it to a Pull Request
- Invite a collaborator to a repository
- Review a Pull Request: comment, request changes, approve
- Merge the Pull Request and see the Issue close automatically

Practical Tasks

Task 6.1 — Author creates the repository; Reviewer joins (4 min)

1. **Author:** on GitHub create a **public** repository `paytm-bill-reminders` with **Add a README file** ticked.
2. **Author:** open **Settings**, **Collaborators**, **Add people**. Type the Reviewer's GitHub username and send the invitation.
3. **Reviewer:** open the invitation (check e-mail or the GitHub notifications bell) and click **Accept invitation**.
4. **Both:** clone the repository.

```
cd ~/git-labs
git clone https://github.com/<author-username>/paytm-bill-reminders.git
cd paytm-bill-reminders
```

Task 6.2 — Author creates an Issue (3 min)

1. On GitHub open the **Issues** tab and click **New issue**.
2. Title: `Add electricity bill reminder`. Description: `We need a reminder file with the electricity bill and its due date.`
3. Click **Submit new issue**. Note the issue number, for example **#1**.

© Akshat Gupta — All Rights Reserved

Task 6.3 — Author builds the change on a branch and pushes it (4 min)

```
git switch -c feature/electricity-reminder
echo "Electricity bill reminder" > reminders.txt
git add reminders.txt
git commit -m "feat: add electricity bill reminder (#1)"
git push -u origin feature/electricity-reminder
```

Task 6.4 — Author opens the Pull Request and links the Issue (3 min)

1. On GitHub click **Compare & pull request**.
2. Title: `Add electricity bill reminder`. In the description write exactly: `Closes #1` (use your issue number) and one sentence about the change.
3. On the right under **Reviewers** choose your partner. Click **Create pull request**.

Task 6.5 — Reviewer reviews the code (4 min)

1. Open the Pull Request and click **Files changed**.
2. Hover over the new line, click the blue **+** and write the comment: `Please also add the due date.` Click **Start a review**.
3. Click **Review changes** (top right), choose **Request changes**, and click **Submit review**.

Reviewer checklist	Yes / No	Comment
Does the change match the Issue?		
Is the change small and clear?		
Is the commit message meaningful?		

Task 6.6 — Author fixes the feedback (3 min)

```
echo "Due date: 10th of every month" >> reminders.txt
git add reminders.txt
git commit -m "fix: add due date to reminder"
git push
```

Open the Pull Request again. The new commit appears in the same PR automatically.

Task 6.7 — Reviewer approves; Author merges (4 min)

1. **Reviewer: Files changed**, **Review changes**, choose **Approve**, **Submit review**.
2. **Author:** go to the **Conversation** tab, click **Merge pull request**, then **Confirm merge**, then **Delete branch**.
3. **Both:** open the **Issues** tab. Issue #1 is now closed, because the description said `Closes #1`.
4. **Both:** update your laptop with the commands below.

© Akshat Gupta — All Rights Reserved

```
git switch main
git pull
cat reminders.txt
```

Expected result — check your work

The Pull Request shows one review that requested changes and one that approved.

Issue #1 is closed and linked to the merged Pull Request.

`reminders.txt` on main contains two lines.

If Something Goes Wrong

What you see	What to do
Reviewer sees a 404 page	The invitation is not accepted yet. Open the invitation e-mail or github.com/notifications and accept it.
Cannot approve own Pull Request	A Reviewer must be a different person from the Author. Ask your partner.
Issue did not close	The PR description must contain `Closes #<number>` and the PR must be merged into main.
git push says rejected after the review	Run `git pull` on your branch first, then `git push` again.

Deliverables

☐	Links to the Issue and the Pull Request
☐	Screenshot of the review conversation
☐	Completed reviewer checklist

Reflection Questions

- Why is it useful that the PR says `Closes #1`?
- What is the difference between Comment, Approve and Request changes?
- What did the Reviewer see in the Files changed tab that helped them review?

© Akshat Gupta — All Rights Reserved

Case Study 7 — Sale Banner: Fixing a Conflict in a Pull Request

Company	Module	Time	Level
Myntra	Module 3 and 4 (Day 3 to 4)	25 minutes	Intermediate

> **Business scenario — Myntra**
> Two Myntra developers change the same line of the sale banner at the same time and each opens a Pull Request. The first one is merged. The second one now shows: **This branch has conflicts that must be resolved**. Git cannot choose between two different versions of the same line, so a human must. You will create this situation yourself and fix it step by step. You can do the whole lab alone.

Learning Objectives

- Understand why a merge conflict happens
- Read conflict markers in a file
- Resolve a conflict and complete the merge
- Get a blocked Pull Request mergeable again

Practical Tasks

Task 7.1 — Create the repository and the banner file (4 min)

1. On GitHub create a **public** repository `myntra-banner` with **Add a README file** ticked.

```
cd ~/git-labs
git clone https://github.com/<username>/myntra-banner.git
cd myntra-banner

cat > banner.txt <<'EOF'
MYNTRA SALE BANNER
Offer: Up to 40% OFF
Valid: 1-7 Nov
EOF

git add banner.txt
git commit -m "feat: add sale banner"
git push
```

© Akshat Gupta — All Rights Reserved

Task 7.2 — Create two branches that change the SAME line (5 min)

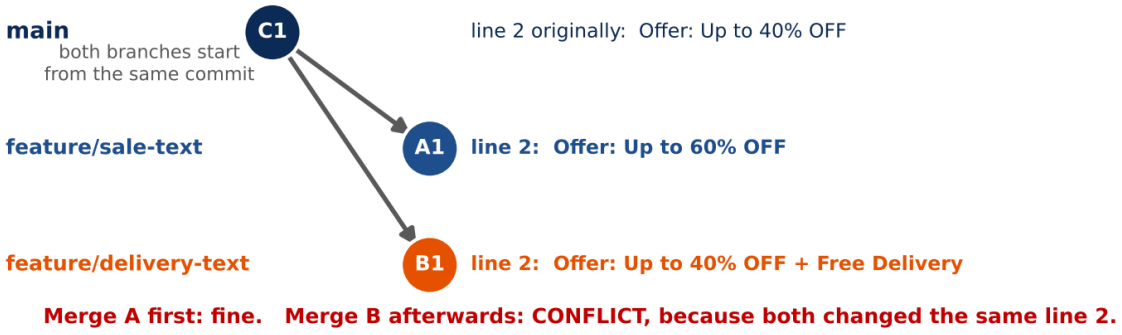


Figure 5: Why this will conflict

Both branches start from the same commit on `main`. The first one changes line 2 to a 60% offer:

```
git switch -c feature/sale-text

cat > banner.txt <<'EOF'
MYNTRA SALE BANNER
Offer: Up to 60% OFF
Valid: 1-7 Nov
EOF

git commit -am "feat: raise offer to 60%"
git push -u origin feature/sale-text
```

Now go **back to main** (very important) and create the second branch, which changes the same line 2 differently:

```
git switch main

git switch -c feature/delivery-text

cat > banner.txt <<'EOF'
MYNTRA SALE BANNER
Offer: Up to 40% OFF + Free Delivery
Valid: 1-7 Nov
EOF

git commit -am "feat: add free delivery to offer"
git push -u origin feature/delivery-text
```

Task 7.3 — Open two Pull Requests (3 min)

1. On GitHub open the **Pull requests** tab and click **New pull request**. Base: `main`, compare: `feature/sale-text`. Create it with the title `Raise offer to 60%`.
2. Repeat for `feature/delivery-text` with the title `Add free delivery`.
3. Open both PRs. Both currently say **This branch has no conflicts with the base branch**.

© Akshat Gupta — All Rights Reserved

Task 7.4 — Merge the first Pull Request and watch the second one break (2 min)

1. Open the PR **Raise offer to 60%** and click **Merge pull request**, **Confirm merge**.
2. Open the PR **Add free delivery**. It now says **This branch has conflicts that must be resolved** and the merge button is disabled.

Task 7.5 — Resolve the conflict on your laptop (9 min)

Step 1. Bring the latest `main` into your second branch:

```
git switch main
git pull
git switch feature/delivery-text
git merge main
```

```
OUTPUT
Auto-merging banner.txt
CONFLICT (content): Merge conflict in banner.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Step 2. See which file is in conflict, then open it in your editor (for example `code banner.txt`):

```
git status
cat banner.txt
```

The file now contains conflict markers:

```
MYNTRA SALE BANNER
<<<<<<< HEAD
Offer: Up to 40% OFF + Free Delivery
=======
Offer: Up to 60% OFF
>>>>>>> main
Valid: 1-7 Nov
```

Marker	Meaning
<<<<<<< HEAD	Start of YOUR version (the branch you are on: feature/delivery-text)
=======	Divider between the two versions
>>>>>>> main	End of the INCOMING version (from main: the 60% offer)

Step 3. Decide what the line should say. We want both changes. Delete the three marker lines and the two old versions, and type one final line, so the file looks exactly like this:

```
MYNTRA SALE BANNER
Offer: Up to 60% OFF + Free Delivery
Valid: 1-7 Nov
```

In VS Code you can also click **Accept Both Changes** and then edit the line. Make sure no `<<<<<<<`, `=======` or `>>>>>>>` text is left.

Step 4. Tell Git the conflict is solved, finish the merge and push:

© Akshat Gupta — All Rights Reserved

```
git add banner.txt
git commit --no-edit
git push
```

```
OUTPUT
[feature/delivery-text 4e5f6a7] Merge branch 'main' into feature/delivery-text
```

Task 7.6 — Merge the second Pull Request and check the result (2 min)

1. Refresh the PR **Add free delivery** on GitHub. The conflict message is gone and **Merge pull request** is green. Merge it.
2. On your laptop run the commands below. The banner shows the combined line.

```
git switch main
git pull
cat banner.txt
```

You can also fix it in the browser

On the conflicted PR click **Resolve conflicts**. GitHub shows the same markers in a web editor. Edit the file, click **Mark as resolved**, then **Commit merge**. Use this for small conflicts; use your laptop for bigger ones.

Expected result — check your work

`banner.txt` on main reads: Offer: Up to 60% OFF + Free Delivery.

Both Pull Requests show the purple **Merged** badge.

`git log --oneline --graph` shows a merge commit.

If Something Goes Wrong

What you see	What to do
A text editor (Vim) opened and I am stuck	Press `Esc`, type `:wq` and press `Enter`. Next time add `--no-edit` to the command.
I do not see CONFLICT; Git says Already up to date	You are on the wrong branch or the first PR is not merged. Run `git branch`, merge PR 1 on GitHub, then repeat from Step 1.
The conflict markers are still in the file	Open the file and delete every line containing <<<<<<<, ======= or >>>>>>>, save, then `git add banner.txt` and `git commit --no-edit`.
I want to cancel and start again	Run `git merge --abort`. Your branch returns to how it was before the merge.
git pull says: you have unmerged files	A merge is still unfinished. Fix the file, then `git add` it and `git commit --no-edit`.

© Akshat Gupta — All Rights Reserved

Deliverables

☐	Links to both Pull Requests
☐	Screenshot of the conflict markers in your editor
☐	Final contents of `banner.txt`
☐	Own words: why did Git ask you to decide?

Reflection Questions

- Why was there NO conflict when the two branches were first created, and a conflict only after the first PR was merged?
- In the markers, which part was yours and which part came from main?
- How can a team reduce the number of conflicts?

© Akshat Gupta — All Rights Reserved

Case Study 8 — Payment Records: Reading History and Undoing Mistakes

Company	Module	Time	Level
Razorpay	Module 2 (Day 2)	20 minutes	Beginner

> **Business scenario — Razorpay**
> A Razorpay developer committed a wrong payment amount (Rs 99999 instead of Rs 999). Before fixing it you need to read the history to find the bad commit, and you also want to know how to undo mistakes safely: edits you have not committed, a wrong commit message, and a whole bad commit.

Learning Objectives

- Read history with git log, git show and git diff
- Undo uncommitted edits and unstage files
- Fix the last commit message with git commit --amend
- Undo a bad commit safely with git revert

Practical Tasks

Task 8.1 — Build the practice repository (4 min)

```
cd ~/git-labs
mkdir razorpay-history
cd razorpay-history
git init

echo "Payment 1: Rs 500" > payments.txt
git add payments.txt
git commit -m "add payment 1"

echo "Payment 2: Rs 750" >> payments.txt
git commit -am "add payment 2"

echo "Payment 3: Rs 1200" >> payments.txt
git commit -am "add payment 3"

echo "Payment 4: Rs 99999" >> payments.txt
git commit -am "add payment 4"
```

Task 8.2 — Read the history (4 min)

```
git log
git log --oneline
git show HEAD
git diff HEAD~1 HEAD
git diff HEAD~3 HEAD
```

© Akshat Gupta — All Rights Reserved

> **Stuck on a screen showing (END)?**
> Press `q` to leave the log or diff view.

`HEAD` means your latest commit and `HEAD~1` the one before it. Answer: which commit added the wrong amount, and what does the green line in `git diff HEAD~1 HEAD` show?

Task 8.3 — Undo changes that are not committed yet (4 min)

```
echo "Payment 5: Rs 300" >> payments.txt
git status
git add payments.txt
git status
git restore --staged payments.txt
git status
git restore payments.txt
git status
cat payments.txt
```

Step by step: `git restore --staged` takes the file OUT of the staging area (edit kept). `git restore` then throws the edit away. Check that Payment 5 is gone.

> **Be careful**
> `git restore <file>` permanently discards edits that were never committed. There is no undo for it.

Task 8.4 — Undo the bad commit with git revert (5 min)

```
git revert HEAD --no-edit
git log --oneline
cat payments.txt
```

```
OUTPUT
[main 7a8b9c0] Revert "add payment 4"
 1 file changed, 1 deletion(-)
```

`git revert` does not erase history. It adds a NEW commit that cancels the bad one. That makes it safe even after the bad commit has been pushed and shared.

Task 8.5 — Add the correct payment and fix a typo in the message (3 min)

```
echo "Payment 4: Rs 999" >> payments.txt
git commit -am "add paymnt 4"
git commit --amend -m "add payment 4"
git log --oneline
```

`--amend` replaces the latest commit, so the typo message is gone. Only amend a commit that you have NOT pushed yet.

© Akshat Gupta — All Rights Reserved

Expected result — check your work

`payments.txt` shows payments 1, 2, 3 and 4 with Rs 999.

`git log --oneline` shows 6 commits, including one starting with `Revert`.

No commit has the typo `paymnt`.

If Something Goes Wrong

What you see	What to do
Screen stuck showing (END) or a colon	Press `q`.
Author identity unknown	Set your name and e-mail (see the setup page), then repeat the commit.
A text editor opened after git revert	Press `Esc`, type `:wq`, press `Enter`. Add `--no-edit` next time.
error: pathspec 'payments.txt' did not match	You are in the wrong folder. `cd ~/git-labs/razorpay-history`.

Deliverables

☐	Output of `git log --oneline`
☐	Answer: which commit added the wrong amount?
☐	Content of `payments.txt` at the end

Reflection Questions

- What is the difference between `git restore --staged` and `git restore`?
- Why is `git revert` safer than deleting a commit when the commit is already on GitHub?
- When is it fine to use `git commit --amend`?

© Akshat Gupta — All Rights Reserved

Case Study 9 — Contributor Wall: Fork a Repository and Send a Pull Request

Company	Module	Time	Level
Ola	Module 4 (Day 4)	20 minutes	Intermediate

> **Business scenario — Ola**
> Ola maintains an open practice repository where contributors add their name. Contributors do not have write access, so they FORK the repository (make their own copy on GitHub), change it, and send a Pull Request back to the owner. You will do exactly this. You need an Owner (your trainer or your partner) and a Contributor (you). Swap roles if time allows.

Learning Objectives

- Explain fork versus clone
- Fork a repository and clone your fork
- Add an upstream remote
- Send a Pull Request from a fork to the original repository

Practical Tasks

Task 9.1 — Owner prepares the practice repository (3 min)

1. **Owner:** create a **public** repository `git-practice` with **Add a README file** ticked.
2. **Owner:** click **Add file**, **Create new file**, type the name `contributors/README.md` (the slash creates the folder), write `Add your own file in this folder.` and click **Commit changes**.
3. **Owner:** share the repository URL with the Contributors.

Task 9.2 — Contributor forks the repository (3 min)

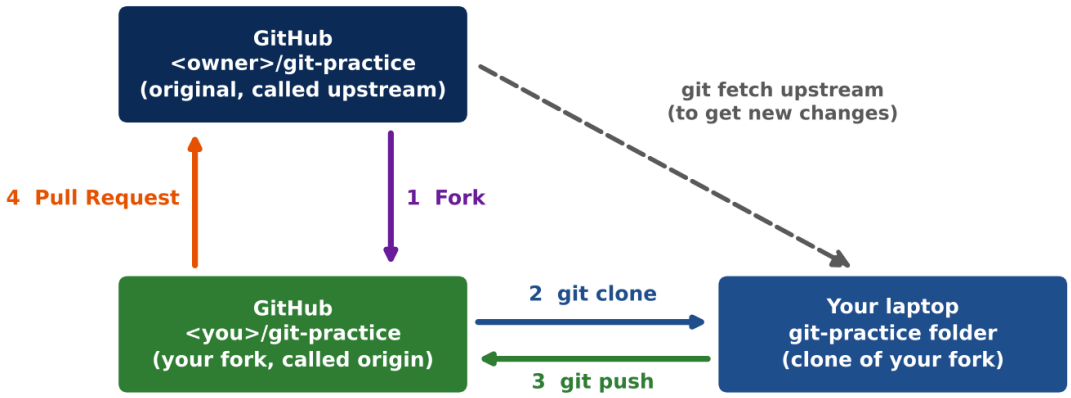


Figure 6: Fork, clone, push and Pull Request

1. Open the Owner's repository page on GitHub.

2. Click **Fork** (top right), then **Create fork**. GitHub opens YOUR copy: `<your-username>/git-practice`.

Task 9.3 — Clone your fork and add the upstream remote (2 min)

```
cd ~/git-labs
git clone https://github.com/<your-username>/git-practice.git
cd git-practice
git remote add upstream https://github.com/<owner-username>/git-practice.git
git remote -v
```

```
OUTPUT
origin    https://github.com/<your-username>/git-practice.git (fetch)
origin    https://github.com/<your-username>/git-practice.git (push)
upstream  https://github.com/<owner-username>/git-practice.git (fetch)
upstream  https://github.com/<owner-username>/git-practice.git (push)
```

`origin` is your fork; `upstream` is the Owner's original repository.

Task 9.4 — Add your file on a branch and push it to your fork (5 min)

Use your first name in lowercase, with no spaces. The example uses `asha`.

```
git switch -c add-asha
echo "Hello, I am Asha" > contributors/asha.txt
git add contributors/asha.txt
git commit -m "docs: add asha to contributors"
git push -u origin add-asha
```

Task 9.5 — Open the Pull Request to the original repository (3 min)

1. On your fork on GitHub click **Compare & pull request**.
2. Check the top of the page: **base repository** must be the OWNER's repository, base branch `main`; **head repository** is your fork with branch `add-asha`.
3. Title: `Add asha to contributors`. Click **Create pull request**.

Task 9.6 — Owner reviews and merges (2 min)

1. **Owner:** open the Pull requests tab, open the PR, check **Files changed**, then click **Merge pull request** and **Confirm merge**.

Task 9.7 — Contributor brings the merged work back into the fork (2 min)

```
git switch main
git fetch upstream
git merge upstream/main
git push origin main
ls contributors
```

Your fork's main is now up to date with the Owner's repository, and `contributors` lists everyone's files.

© Akshat Gupta — All Rights Reserved

Expected result — check your work

The Owner's repository contains `contributors/<name>.txt` for each Contributor.

Your Pull Request shows **Merged**.

`git remote -v` lists both origin and upstream.

If Something Goes Wrong

What you see	What to do
My Pull Request points to my own fork	On the PR page change **base repository** to the Owner's repository.
Which URL do I use for upstream?	The Owner's original repository (not your fork).
git merge upstream/main says: not something we can merge	Run `git fetch upstream` first.
fatal: remote upstream already exists	Check the address with `git remote -v`; fix with `git remote set-url upstream <url>`.

Deliverables

☐	Link to your merged Pull Request
☐	Output of `git remote -v`
☐	Screenshot of the `contributors` folder on the Owner's repository

Reflection Questions

- Why can you not push directly to the Owner's repository?
- What is the difference between origin and upstream?
- When would you fork instead of being added as a collaborator?

© Akshat Gupta — All Rights Reserved

Case Study 10 — Dark-Store Guide: Team Mini Project (Capstone)

Company	Module	Time	Level
Zepto	Module 4 (Day 4)	35 minutes	Intermediate

> **Business scenario — Zepto**
> Three Zepto team members must write a Dark Store Guide together in one file. Each person writes one section on their own branch and raises a Pull Request. Everyone also adds their name to the same Team line, which will cause a conflict that you must fix. This capstone uses the full workflow: Repository, Branch, Development, Commit, Push, Pull Request, Review, Merge.

Learning Objectives

- Follow the complete GitHub team workflow
- Divide work using Issues and branches
- Review each other's Pull Requests
- Resolve a conflict and merge all Pull Requests

Practical Tasks

Task 10.1 — Form the team and create the repository (5 min)

Role	Who and what
Member 1 (Maintainer)	Creates the repository and merges Pull Requests. Writes section 1.
Member 2	Writes section 2.
Member 3	Writes section 3.

1. **Member 1:** create a **public** repository `zepto-store-guide` with a README. Under **Settings, Collaborators** add Member 2 and Member 3.
2. **Members 2 and 3:** accept the invitations.
3. **All three:** clone the repository.

```
cd ~/git-labs
git clone https://github.com/<member1-username>/zepto-store-guide.git
cd zepto-store-guide
```

Task 10.2 — Maintainer adds the starter file (4 min)

Member 1 creates `store-guide.md`, then commits and pushes. **Members 2 and 3** run `git pull` afterwards.

© Akshat Gupta — All Rights Reserved

```
cat > store-guide.md <<'EOF'
# Zepto Dark Store Guide
Team:
## Receiving stock
TODO (Member 1)
## Packing orders
TODO (Member 2)
## Handover to rider
TODO (Member 3)
EOF

git add store-guide.md
git commit -m "docs: add store guide skeleton"
git push


# Members 2 and 3
git pull
ls
```

Task 10.3 — Create the Issues (3 min)

1. **Member 1:** on GitHub open **Issues** and create three Issues: `Write receiving stock section` (#1), `Write packing orders section` (#2), `Write handover to rider section` (#3).
2. Assign each Issue to its owner (Issue #1 to Member 1, and so on) using **Assignees** on the right.

Task 10.4 — Everyone works on their own branch (8 min)

Each member does the following with their own number. Example for **Member 2**:

```
git switch -c feature/packing-orders
```

1. Open `store-guide.md` in your editor.
2. Replace your own `TODO (Member N)` line with two short lines describing your section (for example how orders are packed).
3. Change the line `Team:` to include your name, for example `Team: Ravi`.
4. Save the file.

```
git add store-guide.md
git commit -m "docs: write packing orders section (#2)"
git push -u origin feature/packing-orders
```

> **Branch names**
> Member 1: `feature/receiving-stock`, Member 2: `feature/packing-orders`, Member 3: `feature/rider-handover`.

© Akshat Gupta — All Rights Reserved

Task 10.5 — Open Pull Requests (4 min)

1. Each member opens a Pull Request into `main` from their branch.
2. In the description write `Closes #<your issue number>` and one line about the change.
3. Request a review from a teammate: Member 1 reviews Member 2, Member 2 reviews Member 3, Member 3 reviews Member 1.

Task 10.6 — Review each other's Pull Requests (5 min)

1. Open the Pull Request you were asked to review and go to **Files changed**.
2. Leave at least one comment on a line (for example about spelling or clarity).
3. Click **Review changes**, choose **Approve** (or **Request changes** if something is wrong) and **Submit review**.

Task 10.7 — Merge everything and fix the conflicts (6 min)

1. **Member 1** merges the first approved Pull Request on GitHub.
2. The other Pull Requests now show **This branch has conflicts**, because everyone edited the `Team:` line. Fix them one at a time, **Member 2 first**, using the commands below on your own branch.

```
git switch main
git pull
git switch feature/packing-orders
git merge main
```

Open `store-guide.md`. The conflict is on the `Team:` line. Combine the names so the line reads, for example, `Team: Asha, Ravi`. Remove all marker lines. Then:

```
git add store-guide.md
git commit --no-edit
git push
```

1. Refresh the Pull Request, then **Member 1** merges it.
2. **Member 3** repeats the same steps (now the line should show all three names) and Member 1 merges the last Pull Request.
3. **Everyone:** run the commands below and check the final guide.

```
git switch main
git pull
cat store-guide.md
git log --oneline --graph
```

© Akshat Gupta — All Rights Reserved

Expected result — check your work

`store-guide.md` on main has all three sections written and one Team line with all three names.

Issues #1 to #3 are closed automatically.

All three Pull Requests show Merged and each has at least one review.

If Something Goes Wrong

What you see	What to do
I cannot push to the repository	You have not accepted the collaborator invitation yet. Open github.com/notifications.
The conflict appears again for the third member	That is expected: pull main again, fix the Team line to include all names, add, commit and push.
A text editor opened	Press `Esc`, type `:wq`, press `Enter`.
I broke my file while resolving	Run `git merge --abort` and begin the merge again.

Deliverables

☐	Repository URL
☐	Links to the three Pull Requests
☐	Output of `git log --oneline --graph` on main
☐	Short team note: what went well, what was difficult

Capstone marking rubric (10 marks)

Criterion	Marks	Awarded
Repository, branches and commits with clear messages	2	
Issues created, assigned and linked from the Pull Requests	2	
Pull Request quality: clear title and description	2	
Review quality: comment plus a decision	2	
Conflict resolved correctly, all PRs merged	2	

Reflection Questions

- Why did the Team line conflict while the three sections did not?
- Which step of the workflow stops unreviewed changes from reaching main?
- What would you do differently next time to avoid the conflict?

© Akshat Gupta — All Rights Reserved

Appendix A — Git Command Cheat Sheet

Command	Purpose
`git init`	Create a new local repository
`git clone <url>`	Copy a remote repository with its full history
`git status` / `git status -s`	Show the state of files
`git add <file>` / `git add .`	Stage changes
`git commit -m "msg"`	Record staged changes
`git commit --amend -m "msg"`	Rewrite the latest unpushed commit
`git diff` / `git diff --staged`	Unstaged changes / staged changes
`git restore <file>`	Discard edits that are not committed
`git restore --staged <file>`	Unstage a file
`git revert HEAD --no-edit`	Undo the latest commit by adding a new commit
`git log --oneline --graph --all`	Compact history with branch graph
`git show <rev>`	Show one commit
`git remote add origin <url>`	Connect a remote
`git remote -v`	List remotes and their URLs
`git push -u origin <branch>`	Upload a branch and remember it
`git push -u origin --all`	Upload all local branches
`git fetch` / `git pull`	Download only / download and merge
`git switch <branch>` / `git switch -c <branch>`	Switch to / create and switch to a branch
`git merge <branch>`	Merge a branch into the current branch
`git merge --abort`	Cancel a merge in progress
`git branch -d <branch>`	Delete a merged local branch
`git push origin --delete <branch>`	Delete a branch on GitHub
`git remote add upstream <url>`	Add the original repository of a fork

© Akshat Gupta — All Rights Reserved

Appendix B — Common Errors and How to Fix Them

What you see	What to do
Author identity unknown / Please tell me who you are	Set your name and e-mail with `git config --global user.name` and `git config --global user.email`, then commit again.
Authentication failed / password authentication was removed	Do not type your GitHub password. Use browser sign-in or your Personal Access Token (Option A), or switch to SSH (Option B).
Permission denied (publickey)	SSH key not added to GitHub, or Git is using an HTTPS/SSH URL you did not set up. Redo Option B, or switch the remote with `git remote set-url origin <https-url>`.
fatal: not a git repository	You are in the wrong folder. `cd` into the project folder (check with `pwd` and `ls -a`).
error: src refspec main does not match any	There is no commit yet. `git add .`, `git commit -m "..."`, then push.
remote origin already exists	`git remote set-url origin <correct-url>`.
! [rejected] ... (fetch first)	GitHub has commits you do not have. Run `git pull --no-rebase`, then `git push`.
CONFLICT (content): Merge conflict in <file>	Open the file, keep the lines you want, delete the marker lines, then `git add <file>` and `git commit --no-edit`. To cancel use `git merge --abort`.
A text editor (Vim) opens and will not close	Press `Esc`, type `:wq` and press `Enter`.
Screen stuck with (END) or a colon	Press `q`.
Your local changes would be overwritten by checkout	Commit your work first (`git add .` and `git commit`), then switch branch.
Already up to date (but you expected changes)	Check the branch with `git branch`. You may be on the wrong branch, or the change is not merged on GitHub yet.
I am not sure what state I am in	Run `git status`, `git branch` and `git log --oneline -5`. Then read the hints Git prints.

© Akshat Gupta — All Rights Reserved

Part B — MCQ Assessment (50 Questions)

Instructions

Choose the single best answer for each question. Each correct answer carries 1 mark; there is no negative marking.

Suggested time: 50 minutes (about 1 minute per question). Closed book; the terminal may not be used.

Mark your answers on the answer sheet at the end of this part. Questions are grouped by syllabus module and tagged with the company context.

Module 1 — Introduction to Git & GitHub (Q1 to Q12)

Q1. [Zomato] Forty developers at Zomato work on one codebase. What is the BEST reason to use version control?

- **A.** It makes the application run faster in production
- **B.** It removes the need for code reviews and testing
- **C.** It compresses source files to save disk space
- **D.** It records who changed what and when, and lets the team work in parallel without overwriting each other

Q2. [Swiggy] Which statement about Git is correct?

- **A.** Git is a website that hosts repositories
- **B.** Git is a code editor built by Microsoft
- **C.** Git is a distributed version control system that runs on your own machine
- **D.** Git is a CI/CD server that deploys applications

Q3. [Flipkart] What does GitHub add on top of Git?

- **A.** A faster compression format for commits
- **B.** A replacement for the git commit command
- **C.** Cloud hosting of repositories plus collaboration features such as pull requests and issues
- **D.** A programming language for writing hooks

Q4. [Paytm] Which pair correctly distinguishes Git from GitHub?

- **A.** Git: an online platform; GitHub: a command-line tool
- **B.** Git works only with GitHub; GitHub works with any VCS
- **C.** Git needs the internet for every command; GitHub does not
- **D.** Git: the version control tool on your computer; GitHub: an online platform that hosts Git repositories

Q5. [Razorpay] 'Distributed' in a distributed version control system means:

- **A.** files are automatically split across many servers
- **B.** only the central server stores the history
- **C.** code is stored on a blockchain
- **D.** every developer's clone holds the full repository history

© Akshat Gupta — All Rights Reserved

Q6. [PhonePe] You have an existing folder of source code. Which command turns it into a Git repository?

A. git start
B. git init
C. git create
D. git new

Q7. [Ola] What does the hidden .git folder contain?

A. Only the list of ignored files
B. The compiled build output
C. Your GitHub login password
D. The repository database: commit history, objects and configuration

Q8. [Myntra] What is the purpose of git config --global user.name and user.email?

A. They log you in to your GitHub account
B. They set the author name and e-mail that Git stores in every commit you create
C. They create a new GitHub account for you
D. They choose the name of the default branch

Q9. [Nykaa] What does git remote -v display?

A. The names and URLs of the remote repositories your repository is connected to (fetch and push)
B. The list of all local branches
C. The files that are staged for the next commit
D. The commit history of the remote

Q10. [Zepto] What is the main purpose of README.md in a repository?

A. To store environment variables
B. To list the files Git should ignore
C. To describe the project and explain how to install and use it; GitHub shows it on the repository page
D. To configure Git's merge behaviour

Q11. [Zomato] Which of these is a remote repository?

A. The copy of the project hosted on GitHub, usually named origin
B. The hidden .git folder inside your project
C. Your working directory
D. The staging area

Q12. [Swiggy] What is the correct order of the basic Git workflow for saving new work locally?

A. git commit, then git add, then edit files
B. Edit files, then git add, then git commit
C. git add, then edit files, then git commit
D. git push, then git add, then git commit

© Akshat Gupta — All Rights Reserved

Module 2 — Git Basics & Repository Management (Q13 to Q28)

Q13. [Flipkart] Which command moves changes from the working directory into the staging area?

A. git commit
B. git push
C. git clone
D. git add

Q14. [Paytm] After you edit a tracked file, where does git status list it?

A. Changes to be committed
B. Untracked files
C. Changes not staged for commit
D. Ignored files

Q15. [Razorpay] What does a plain git diff (no options) show?

A. Differences between the working directory and the staging area
B. Differences between the staging area and the last commit
C. Differences between local and remote branches
D. The full commit history

Q16. [PhonePe] What does git diff --staged show?

A. Edits that are not yet staged
B. Differences between staged changes and the last commit
C. Differences between local and remote
D. The list of all branches

Q17. [Ola] Which command prints the commit history with one line per commit?

A. git history --short
B. git status -s
C. git log --oneline
D. git show --list

Q18. [Myntra] How does Git uniquely identify each commit?

A. A sequential number: 1, 2, 3...
B. The commit message text
C. The name of the branch it was made on
D. A unique hash ID (a SHA-1 hash by default)

Q19. [Nykaa] You staged cart.py by mistake but want to keep your edits. Which command unstages it?

A. git rm cart.py
B. git restore --staged cart.py
C. git reset --hard
D. git commit --amend

© Akshat Gupta — All Rights Reserved

Q20. [Zepto] **Which command discards uncommitted edits in cart.py and restores the last committed version?**

- **A.** git add cart.py
- **B.** git restore cart.py
- **C.** git stash pop
- **D.** git commit --amend

Q21. [Zomato] **Which command deletes a tracked file and stages the deletion in one step?**

- **A.** git rm old_utils.py
- **B.** rm old_utils.py
- **C.** git delete old_utils.py
- **D.** git remove --staged old_utils.py

Q22. [Swiggy] **In git status -s output, what does ?? beside a filename mean?**

- **A.** The file has a merge conflict
- **B.** The file is ignored
- **C.** The file is untracked
- **D.** The file was deleted

Q23. [Flipkart] **You made a typo in the message of your latest commit, which has not been pushed. How do you fix it?**

- **A.** git commit --amend -m "corrected message"
- **B.** git revert HEAD
- **C.** git reset --hard origin/main
- **D.** git rename-commit HEAD

Q24. [Paytm] **Which command renames a tracked file and stages the rename?**

- **A.** git rename old.py new.py
- **B.** git mv old.py new.py
- **C.** git move old.py new.py
- **D.** git tag old.py new.py

Q25. [Razorpay] **What does git show <hash> display?**

- **A.** Only the remote URL
- **B.** All branches in the repository
- **C.** The commit's metadata and the changes it introduced
- **D.** Files currently in the staging area

Q26. [PhonePe] **Which commit message follows good practice?**

- **A.** fix: round UPI refund amount to two decimals
- **B.** changes
- **C.** fixed stuff
- **D.** asdf123

© Akshat Gupta — All Rights Reserved

Q27. [Ola] **Which command connects a local repository to a GitHub repository and names it origin?**

- **A.** git link origin <url>
- **B.** git remote add origin <url>
- **C.** git connect <url> origin
- **D.** git origin add <url>

Q28. [Myntra] **What does the -u option do in git push -u origin main?**

- **A.** Forces the push to overwrite remote history
- **B.** Sets the upstream so later git push and git pull work without arguments
- **C.** Uploads only untracked files
- **D.** Unstages all files before pushing

Module 3 — GitHub & Collaboration (Q29 to Q40)

Q29. [Nykaa] **What does git clone <url> do?**

- **A.** Copies the repository with its full history and sets up origin
- **B.** Downloads only the latest files without history
- **C.** Uploads your local commits to GitHub
- **D.** Creates a new branch called clone

Q30. [Zepto] **What is the difference between git fetch and git pull?**

- **A.** fetch merges changes; pull only downloads
- **B.** They are identical
- **C.** fetch uploads local commits; pull downloads them
- **D.** fetch downloads remote changes without merging; pull is fetch followed by merge

Q31. [Zomato] **git push is rejected with 'fetch first' (non-fast-forward). What does this usually mean?**

- **A.** Your README.md is missing
- **B.** The remote has commits you do not have locally
- **C.** GitHub is out of storage
- **D.** You have too many local branches

Q32. [Swiggy] **Why do teams use feature branches?**

- **A.** To make the repository smaller
- **B.** To isolate work in progress from stable code so people can develop in parallel
- **C.** To hide code from other developers
- **D.** To avoid writing commit messages

Q33. [Flipkart] **Which command creates a new branch and switches to it?**

- **A.** git switch -c feature/offers
- **B.** git branch -d feature/offers
- **C.** git merge feature/offers
- **D.** git clone feature/offers

© Akshat Gupta — All Rights Reserved

Q34. [Paytm] **When does Git perform a fast-forward merge?**

- **A.** When both branches edited the same line
- **B.** When the target branch has no new commits since the feature branch was created
- **C.** When the feature branch is deleted first
- **D.** When a merge conflict exists

Q35. [Razorpay] **You are on feature/upi-qr and want to merge it into main. Which sequence is correct?**

- **A.** git switch feature/upi-qr, then git merge main
- **B.** git push main, then git merge origin
- **C.** git switch main, then git merge feature/upi-qr
- **D.** git merge --abort, then git switch main

Q36. [PhonePe] **In a conflicted file, what appears between <<<<<<< HEAD and =======?**

- **A.** The version from the branch being merged in
- **B.** The common ancestor version
- **C.** Lines deleted by both branches
- **D.** The version from the branch you are currently on

Q37. [Ola] **After you edit a conflicted file and remove the conflict markers, what finishes the merge?**

- **A.** git push --force
- **B.** git branch -d
- **C.** git clone
- **D.** git add <file>, then git commit

Q38. [Myntra] **A merge stops with CONFLICT and you want to go back to the state before the merge. Which command?**

- **A.** git merge --abort
- **B.** git commit --amend
- **C.** git clean -f
- **D.** git pull --rebase

Q39. [Nykaa] **Which command safely deletes a local branch that has already been merged?**

- **A.** git branch -d feature/offers
- **B.** git rm branch feature/offers
- **C.** git delete feature/offers
- **D.** git clone -d feature/offers

Q40. [Zepto] **When does a merge conflict occur?**

- **A.** Whenever two branches are merged
- **B.** When both branches change the same lines of a file in different ways
- **C.** When two different files are edited
- **D.** When branch names are similar

© Akshat Gupta — All Rights Reserved

Module 4 — GitHub Team Workflow (Q41 to Q50)

Q41. [Zomato] **What is a Pull Request?**

- **A.** A command that downloads remote commits
- **B.** A request to GitHub support to restore a repository
- **C.** A proposal to merge changes from one branch or fork into another, with room for review and discussion
- **D.** A backup of the repository

Q42. [Swiggy] **Which sequence matches the standard GitHub team workflow?**

- **A.** Branch, commit, push, open pull request, review, merge
- **B.** Merge, review, open pull request, branch
- **C.** Push to main, review, create branch
- **D.** Open pull request, clone, commit, fork

Q43. [Flipkart] **What does writing 'Closes #12' in a pull request description do?**

- **A.** Deletes branch number 12
- **B.** Merges pull request #12
- **C.** Locks the conversation on issue #12
- **D.** Closes issue #12 automatically when the PR is merged into the default branch

Q44. [Paytm] **A reviewer selects 'Request changes' on a pull request. What does that signal?**

- **A.** The reviewer wants changes made before the PR should be approved
- **B.** The PR is merged immediately
- **C.** The PR is permanently closed
- **D.** The author may approve their own PR

Q45. [Razorpay] **What is the difference between fork and clone?**

- **A.** A fork is on your laptop; a clone is on GitHub
- **B.** They are the same operation
- **C.** Forking deletes the original repository
- **D.** A fork is a server-side copy under your GitHub account; a clone is a local copy on your machine

Q46. [PhonePe] **You forked an open-source project and want your fork to get the original project's new commits. What do you do?**

- **A.** Delete the fork and create it again
- **B.** Run git init inside the fork
- **C.** Nothing: forks always update themselves automatically
- **D.** Add the original repository as a remote (commonly upstream) and fetch from it

Q47. [Ola] **What are GitHub Issues mainly used for?**

- **A.** Running automated tests
- **B.** Storing compiled binaries
- **C.** Tracking bugs, tasks and feature requests with discussion
- **D.** Encrypting repository secrets

© Akshat Gupta — All Rights Reserved

Q48. [Myntra] **Which statement about public and private repositories is correct?**

- **A.** Private repositories are visible to all GitHub users
- **B.** Public repositories cannot be forked
- **C.** Public repositories are visible to everyone; private ones only to people you grant access to
- **D.** Visibility can never be changed after creation

Q49. [Nykaa] **Which merge option on GitHub combines all commits of a PR into a single commit on the base branch?**

- **A.** Squash and merge
- **B.** Create a merge commit
- **C.** Rebase and merge
- **D.** Close pull request

Q50. [Zepto] **Two PRs edit the same line of README.md. The first is merged and the second now shows a conflict. What is the best next step?**

- **A.** Delete the second PR and discard the work
- **B.** Force-merge the PR ignoring the conflict
- **C.** Update the second branch with main, resolve the conflict, push, and then merge
- **D.** Fork the repository again

© Akshat Gupta — All Rights Reserved

Answer Sheet

Write A, B, C or D beside each question number. Detach this page before the trainer's key is shown.

Q	Answer	Q	Answer	Q	Answer	Q	Answer	Q	Answer
Q1		Q11		Q21		Q31		Q41	
Q2		Q12		Q22		Q32		Q42	
Q3		Q13		Q23		Q33		Q43	
Q4		Q14		Q24		Q34		Q44	
Q5		Q15		Q25		Q35		Q45	
Q6		Q16		Q26		Q36		Q46	
Q7		Q17		Q27		Q37		Q47	
Q8		Q18		Q28		Q38		Q48	
Q9		Q19		Q29		Q39		Q49	
Q10		Q20		Q30		Q40		Q50	

Name	
Batch / Date	
Score (out of 50)	

© Akshat Gupta — All Rights Reserved

Part C — Instructor Answer Key

For the trainer. Remove this section before distributing the workbook to participants.

Q	Ans	Explanation
1	D	Version control keeps a full, attributable history and enables safe parallel work. It does not change runtime speed or replace testing.
2	C	Git is the tool installed locally. Hosting services such as GitHub are separate from Git itself.
3	C	GitHub is a hosting platform built around Git repositories; pull requests, issues and access control are GitHub features.
4	D	Most Git commands (add, commit, log, branch) work offline. GitHub is one of several hosts (GitLab and Bitbucket are others).
5	D	In Git each clone is a complete repository with full history, so most work continues even when offline.
6	B	git init creates the hidden .git directory that stores the repository database.
7	D	Deleting .git removes the project's history; the working files remain but are no longer under version control.
8	B	Git records an author with each commit, so the identity must be configured once per machine. It is not a GitHub login.
9	A	git remote -v is the quickest way to check where origin points, for example when a push goes to the wrong repository.
10	C	README.md is the project's front page and documentation entry point.
11	A	A remote repository lives on another machine or service. The .git folder is your local repository.
12	B	You change files in the working directory, stage them with git add, and record them with git commit.
13	D	git add stages changes; git commit then records the staged snapshot in the local repository.
14	C	A modified tracked file is unstaged until you run git add.
15	A	git diff compares working directory to the index; git diff --staged compares the index to HEAD.
16	B	--staged (alias --cached) previews exactly what the next commit will contain.
17	C	git log --oneline shows the abbreviated hash and subject line of each commit.
18	D	The hash is computed from the commit's content and metadata, so it is unique and tamper-evident.
19	B	git restore --staged moves the file back out of the staging area without touching your edits. git reset --hard would discard work.
20	B	git restore overwrites the working copy with the staged/committed version. Unstaged edits cannot be recovered afterwards, so use it carefully.
21	A	A plain rm only deletes the file from the working directory; the deletion still has to be staged. git rm does both.
22	C	?? marks untracked files. Conflicts show as UU and deletions as D.

© Akshat Gupta — All Rights Reserved

Q	Ans	Explanation
23	A	--amend rewrites the most recent commit. Avoid amending commits that are already pushed and shared.
24	B	git mv is shorthand for renaming the file and staging the change; Git detects it as a rename.
25	C	git show prints author, date, message and the diff for the given commit.
26	A	A good message is specific and explains what changed and why, which makes git log useful during audits.
27	B	git remote add <name> <url> registers a remote; origin is the conventional name for the main one.
28	B	-u (--set-upstream) links the local branch to origin/main for future pushes and pulls.
29	A	A clone is a complete local copy with a remote named origin pointing to the source.
30	D	fetch is the safe way to look at what changed on the remote before integrating it.
31	B	Integrate the remote work first with git pull (or fetch and merge/rebase), then push again.
32	B	Branches are cheap pointers that keep unfinished work away from the stable main branch.
33	A	git switch -c (or git checkout -b) creates and checks out the branch in one step.
34	B	If main has not moved, Git simply moves the main pointer forward; no merge commit is needed.
35	C	git merge brings the named branch into the branch you are currently on, so first switch to the receiving branch (main).
36	D	The section after ======= up to >>>>>>> is the incoming branch's version.
37	D	Staging the file marks the conflict as resolved; the commit then completes the merge.
38	A	git merge --abort restores the pre-merge state while the merge is still in progress.
39	A	-d refuses to delete an unmerged branch; -D forces deletion and can lose commits.
40	B	Changes in different files or different regions of a file merge automatically.
41	C	PRs are the unit of collaboration on GitHub: they carry the diff, review comments, checks and the merge decision.
42	A	This is the Repository, Branch, Development, Commit, Push, Pull Request, Review, Merge flow used in the course project.
43	D	GitHub closing keywords (Closes, Fixes, Resolves) link a PR to an issue and close it on merge.
44	A	The author pushes more commits to the same branch and the reviewer re-reviews.
45	D	Typical open-source flow: fork on GitHub, clone your fork locally, push to your fork, open a PR to the original.
46	D	git remote add upstream <original-url>, then git fetch upstream and merge into your branch.
47	C	Issues can be labelled, assigned and linked to pull requests.
48	C	Owners can change visibility later in Settings (with some consequences, for example for forks).
49	A	Squash and merge keeps the base branch history short, with one commit per PR.

© Akshat Gupta — All Rights Reserved

Q	Ans	Explanation
50	C	Bring main into the feature branch (merge or rebase), fix the conflicting lines, push, and the PR becomes mergeable.

Marking Scheme

Case study labs (10 marks each, 100 in total)

Criterion	Marks	How to award
Correct execution of all tasks	5	Commands run, repository state matches the Expected result box
Evidence submitted	3	Required screenshots, outputs and links are present and legible
Reflection questions	2	Answers show understanding, not copied commands

The Zepto capstone (Case Study 10) uses its own rubric, shown at the end of that case study, which also totals 10 marks.

MCQ assessment (50 marks)

Score	Result	Suggested action
45 to 50	Distinction	Ready to mentor peers
38 to 44	Merit	Solid understanding; review missed questions
30 to 37	Pass	Revisit modules where more than two questions were missed
Below 30	Needs revision	Repeat the relevant labs and retake the quiz

Overall result (150 marks)

Labs 100 + MCQs 50. Pass mark: 60 percent (90 marks). Distinction: 85 percent (128 marks).

Question distribution by module

Module	Questions	Range
Module 1 — Introduction to Git & GitHub	12	Q1 to Q12
Module 2 — Git Basics & Repository Management	16	Q13 to Q28
Module 3 — GitHub & Collaboration	12	Q29 to Q40
Module 4 — GitHub Team Workflow	10	Q41 to Q50

© Akshat Gupta — All Rights Reserved